

CRAN Task View: High-Performance and Parallel Computing with R

Maintainer: Dirk Eddelbuettel

Contact: Dirk.Eddelbuettel at R-project.org

Version: 2023-04-29

URL: <https://CRAN.R-project.org/view=HighPerformanceComputing>

Source: <https://github.com/cran-task-views/HighPerformanceComputing/>

Contributions: Suggestions and improvements for this task view are very welcome and can be made through issues or pull requests on GitHub or via e-mail to the maintainer address. For further details see the [Contributing guide](#).

Citation: Dirk Eddelbuettel (2023). CRAN Task View: High-Performance and Parallel Computing with R. Version 2023-04-29. URL <https://CRAN.R-project.org/view=HighPerformanceComputing>.

Installation: The packages from this task view can be installed automatically using the [ctv](#) package. For example, `ctv::install.views("HighPerformanceComputing", coreOnly = TRUE)` installs all the core packages or `ctv::update.views("HighPerformanceComputing")` installs all packages that are not yet installed and up-to-date. See the [CRAN Task View Initiative](#) for more details.

This CRAN Task View contains a list of packages, grouped by topic, that are useful for high-performance computing (HPC) with R. In this context, we are defining ‘high-performance computing’ rather loosely as just about anything related to pushing R a little further: using compiled code, parallel computing (in both explicit and implicit modes), working with large objects as well as profiling.

Unless otherwise mentioned, all packages presented with hyperlinks are available from the [Comprehensive R Archive Network \(CRAN\)](#).

Several of the areas discussed in this Task View are undergoing rapid change. Please send suggestions for additions and extensions for this task view via e-mail to the maintainer or submit an issue or pull request in the GitHub repository linked above. See the [Contributing page](#) in the [CRAN Task Views](#) repo for details.

Suggestions and corrections by Achim Zeileis, Markus Schmidberger, Martin Morgan, Max Kuhn, Tomas Radiovoyevitch, Jochen Knaus, Tobias Verbeke, Hao Yu, David Rosenberg, Marco Enea, Ivo Welch, Jay Emerson, Wei-Chen Chen, Bill Cleveland, Ross Boylan, Ramon Diaz-Uriarte, Mark Zeligman, Kevin Ushey, Graham Jeffries, Will Landau, Tim Flutre, Reza Mohammadi, Ralf Stubner, Bob Jansen, Matt Fidler, Brent Brewington and Ben Bolder (as well as others I may have forgotten to add here) are gratefully acknowledged.

The `ctv` package supports these Task Views. Its functions `install.views` and `update.views` allow, respectively, installation or update of packages from a given Task View; the option `coreOnly` can restrict operations to packages labeled as *core* below.

Direct support in R started with release 2.14.0 which includes a new package **parallel** incorporating (slightly revised) copies of packages `multicore` and [snow](#). Some types of clusters are not handled directly by the base package ‘parallel’. However, and as explained in the package vignette, the parts of `parallel` which provide [snow](#)-like functions will accept [snow](#) clusters including MPI clusters. Use `vignette("parallel")` to view the package vignette. The **parallel** package also contains support for multiple RNG streams following L’Ecuyer et al (2002), with support for both `mclapply` and `snow` clusters. The version released for R 2.14.0 contains base functionality: higher-level convenience functions are planned for later R releases.

Parallel computing: Explicit parallelism

- Several packages provide the communications layer required for parallel computing. The first package in this area was `rpvm` by Li and Rossini which uses the PVM (Parallel Virtual Machine) standard and libraries. `rpvm` is no longer actively maintained, but available from its CRAN archive directory.
- In recent years, the alternative MPI (Message Passing Interface) standard has become the de facto standard in parallel computing. It is supported in R via the [Rmpi](#) by Yu. [Rmpi](#) package is mature yet actively maintained and offers access to numerous functions from the MPI API, as well as a number of R-specific extensions. [Rmpi](#) can be used with the LAM/MPI, MPICH / MPICH2, Open MPI, and Deino MPI implementations. It should be noted that LAM/MPI is now in maintenance mode, and new development is focused on Open MPI.
- The [pbdMPI](#) package provides S4 classes to directly interface MPI in order to support the Single Program/Multiple Data (SPMD) parallel programming style which is particularly useful for batch parallel

execution.

- The [snow](#) (Simple Network of Workstations) package by Tierney et al. can use PVM, MPI, NWS as well as direct networking sockets. It provides an abstraction layer by hiding the communications details. The [snowFT](#) package provides fault-tolerance extensions to [snow](#).
- The [snowfall](#) package by Knaus provides a more recent alternative to [snow](#). Functions can be used in sequential or parallel mode.
- The [parallelly](#) package enhances the parallel package by giving additional control over launch and set-up of parallel workers.
- The [foreach](#) package allows general iteration over elements in a collection without the use of an explicit loop counter. Using foreach without side effects also facilitates executing the loop in parallel which is possible via the [doMC](#) (using parallel/multicore on single workstations), [doSNOW](#) (using [snow](#), see above), [doMPI](#) (using [Rmpi](#)) packages, and [doFuture](#) (using [future](#)) packages.
- The [future](#) package allows for synchronous (sequential) and asynchronous (parallel) evaluations via abstraction of futures, either via function calls or implicitly via promises. Global variables are automatically identified. Iteration over elements in a collection is supported. Parallel map-reduce calls via the future framework are provided by packages [future.apply](#) for parallel versions of base-R apply functions, and [furrr](#) for parallel versions of purrr functions. Parallelization is available through the parallel package, [future.callr](#) via the callr package, and [future.batchtools](#) via the batchtools package.
- The [Rborist](#) package employs OpenMP pragmas to exploit predictor-level parallelism in the Random Forest algorithm which promotes efficient use of multicore hardware in restaging data and in determining splitting criteria, both of which are performance bottlenecks in the algorithm.
- The [h2o](#) package connects to the h2o open source machine learning environment which has scalable implementations of random forests, GBM, GLM (with elastic net regularization), and deep learning.
- The [randomForestSRC](#) package can use both OpenMP as well as MPI for random forest extensions suitable for survival analysis, competing risks analysis, classification as well as regression
- The [parSim](#) package can perform simulation studies using one or multiple cores, both locally and on HPC clusters.
- The [qsub](#) package can submit commands to run on gridengine clusters.
- The [mirai](#) package is a minimalist framework for local or distributed asynchronous code evaluation, implementing futures which automatically resolve upon completion, built on the high-performance [nanonext](#) NNG C messaging library binding.
- The [condor](#) package can interact with Condor HPC installations via ssh to transfer files and access remote compute jobs.

Parallel computing: Implicit parallelism

- The pnmath package by Tierney ([link](#)) uses the OpenMP parallel processing directives of recent compilers (such gcc 4.2 or later) for implicit parallelism by replacing a number of internal R functions with replacements that can make use of multiple cores --- without any explicit requests from the user. The alternate pnmath0 package offers the same functionality using Pthreads for environments in which the newer compilers are not available. Similar functionality is expected to become integrated into R 'eventually'.
- The romp package by Jamitzky was presented at useR! 2008 ([slides](#)) and offers another interface to OpenMP using Fortran. The code is still pre-alpha and available from the Google Code project [romp](#). An R-Forge project [romp](#) was initiated but there is no package, yet.
- The [RhpcBLASctl](#) package detects the number of available BLAS cores, and permits explicit selection of the number of cores.
- The [targets](#) package and its predecessor [drake](#) are R-focused pipeline toolkits similar to [Make](#) . Each constructs a directed acyclic graph representation of the workflow and orchestrates distributed computing across `clustermq` and future workers.
- The [flexiblas](#) package manages BLAS/LAPACK libraries by loading and possibly switching them if FlexiBLAS ([link](#)) is used.

Parallel computing: Grid computing

- The multiR package by Grose was presented at useR! 2008 but has not been released. It may offer a snow-style framework on a grid computing platform.
- The [biocep-distrib](#) project by Chine offers a Java-based framework for local, Grid, or Cloud computing. It is under active development.

Parallel computing: Hadoop

- The [RHIPE](#) package, started by Saptarshi Guha, provides an interface between R and Hadoop for analysis of large complex data wholly from within R using the Divide and Recombine approach to big data.
- The [rmr](#) package by Revolution Analytics also provides an interface between R and Hadoop for a Map/Reduce programming framework. ([link](#))
- A related package, [segue](#) package by Long, permits easy execution of embarrassingly parallel task on Elastic Map Reduce (EMR) at Amazon. ([link](#))
- The [RProtoBuf](#) package provides an interface to Google's language-neutral, platform-neutral, extensible mechanism for serializing structured data. This package can be used in R code to read data streams from other systems in a distributed MapReduce setting where data is serialized and passed back and forth between tasks.
- The [HistogramTools](#) package provides a number of routines useful for the construction, aggregation, manipulation, and plotting of large numbers of histograms such as those created by mappers in a MapReduce application.

Parallel computing: Random numbers

- Random-number generators for parallel computing are available via the [rlecuyer](#) package, the [rstream](#) package, the [sitmo](#) package as well as the [dqrng](#) package.
- The [doRNG](#) package provides functions to perform reproducible parallel foreach loops, using independent random streams as generated by the package [rstream](#), suitable for the different foreach backends.

Parallel computing: Resource managers and batch schedulers

- Job-scheduling toolkits permit management of parallel computing resources and tasks. The [slurm](#) (Simple Linux Utility for Resource Management) set of programs works well with MPI and [slurm](#) jobs can be submitted from R using the [rslurm](#) package. ([link](#))
- The [Condor](#) toolkit ([link](#)) from the University of Wisconsin-Madison has been used with R as described in this [R News article](#) .
- The [sfCluster](#) package by Knaus can be used with [snowfall](#). ([link](#)) but is currently limited to LAM/MPI.
- The [batch](#) package by Hoffmann can launch parallel computing requests onto a cluster and gather results.
- The [BatchJobs](#) package provides Map, Reduce and Filter variants to manage R jobs and their results on batch computing systems like PBS/Torque, LSF and Sun Grid Engine. Multicore and SSH systems are also supported. The [BatchExperiments](#) package extends it with an abstraction layer for running statistical experiments. Package [batchtools](#) is a successor / extension to both.
- The [flowr](#) package offers a scatter-gather approach to submit jobs lists (including dependencies) to the computing cluster via simple data.frames as inputs. It supports LSF, SGE, Torque and SLURM.
- The [clustermq](#) package sends function calls as jobs on LSF, SGE and SLURM via a single line of code without using network-mounted storage. It also supports use of remote clusters via SSH.

Parallel computing: Applications

- The [caret](#) package by Kuhn can use various frameworks (MPI, NWS etc) to parallelized cross-validation and bootstrap characterizations of predictive models.
- The [maanova](#) package on Bioconductor by Wu can use [snow](#) and [Rmpi](#) for the analysis of micro-array experiments.
- The [pvcust](#) package by Suzuki and Shimodaira can use [snow](#) and [Rmpi](#) for hierarchical clustering via multiscale bootstraps.
- The [tm](#) package by Feinerer can use [snow](#) and [Rmpi](#) for parallelized text mining.
- The [varSelRF](#) package by Diaz-Uriarte can use [snow](#) and [Rmpi](#) for parallelized use of variable selection via random forests.
- The [bcp](#) package by Erdman and Emerson for the Bayesian analysis of change points can use [foreach](#) for parallelized operations.
- The [multtest](#) package by Pollard et al. on Bioconductor can use [snow](#), [Rmpi](#) or [rpvm](#) for resampling-based testing of multiple hypothesis.
- The [Matching](#) package by Sekhon for multivariate and propensity score matching, the [bnlearn](#) package by Scutari for bayesian network structure learning, the [latentnet](#) package by Krivitsky and Handcock for latent position and cluster models, the [peperr](#) package by Porzelius and Binder for parallelised estimation of prediction error, the [orloca](#) package by Fernandez-Palacin and Munoz-Marquez for operations research locational analysis, the [rgenoud](#) package by Mebane and Sekhon for genetic optimization using derivatives, the [affyPara](#) package by Schmidberger, Vicedo and Mansmann for parallel normalization of Affymetrix microarrays, and the [puma](#) package by Pearson et al. which propagates uncertainty into standard microarray analyses such as differential

expression all can use [snow](#) for parallelized operations using either one of the MPI, PVM, NWS or socket protocols supported by [snow](#).

- The [bugsparell](#) package uses [Rmpi](#) for distributed computing of multiple MCMC chains using WinBUGS.
- The [xgboost](#) package by Chen et al. is an optimized distributed gradient boosting library designed to be highly efficient, flexible and portable. The same code runs on major distributed environment, such as Hadoop, SGE, and MPI.
- The [dclone](#) package provides a global optimization approach and a variant of simulated annealing which exploits Bayesian MCMC tools to get MLE point estimates and standard errors using low level functions for implementing maximum likelihood estimating procedures for complex models using data cloning and Bayesian Markov chain Monte Carlo methods with support for JAGS, WinBUGS and OpenBUGS; parallel computing is supported via the [snow](#) package.
- Nowadays, many packages can use the facilities offered by the **parallel** package. One example is [pls](#).
- The [pbapply](#) package offers a progress bar for vectorized R functions in the `\*apply` family, and supports several backends.
- The [Sim.DiffProc](#) package simulates and estimates multidimensional Itô and Stratonovich stochastic differential equations in parallel.
- The [keras](#) package by by Allaire et al. provides a high-level neural networks API. It was developed with a focus on enabling fast experimentation for convolutional networks, recurrent networks, any combination of both, and custom neural network architectures.
- The [mvnfast](#) uses the sumo random number generator to generate multivariate and normal distributions in parallel.
- The [rxode2random](#) uses the [sitmo](#) package to generate either truncated or non-truncated multivariate normal distributions in parallel. The package also generates many other common distributions in parallel (like binomial, t-distribution etc).
- The [rxode2](#) uses parallel processing (via OpenMP) for faster solving of ordinary differential equations (ODEs) over multiple units (grouped by ID) and can generate random numbers for each ODE simulation problem (done automatically with the support package [rxode2random](#)).
- The [nlmixr2](#) uses parallel ODE solving from rxode2 to solve nonlinear mixed effects models in parallel (for the algorithm "saem").

Parallel computing: GPUs

- The [rgpu](#) package (see below for link) aims to speed up bioinformatics analysis by using the GPU.
- The [gcbd](#) package implements a benchmarking framework for BLAS and GPUs.
- The [OpenCL](#) package provides an interface from R to OpenCL permitting hardware- and vendor neutral interfaces to GPU programming.
- The [tensorflow](#) package by by Allaire et al. provides access to the complete TensorFlow API from within R that enables numerical computation using data flow graphs. The flexible architecture allows users to deploy computation to one or more CPUs or GPUs in a desktop, server, or mobile device with a single API.
- The [tfestimators](#) package by by Tang et al. offers a high-level API that provides implementations of many different model types including linear models and deep neural networks. It also provides a flexible framework for defining arbitrary new model types as custom estimators with the distributed power of TensorFlow for free.
- The [BDgraph](#) package provides statistical tools for Bayesian structure learning in undirected graphical models for multivariate continuous, discrete, and mixed data using parallel sampling algorithms implemented using OpenMP and C++.
- The [ssgraph](#) package offers Bayesian inference in undirected graphical models using spike-and-slab priors for multivariate continuous, discrete, and mixed data. Computationally intensive tasks of the package are using OpenMP via C++.
- The [GPUmatrix](#) package can offload calculations to the GPU while providing the API of the `Matrix` package.

Large memory and out-of-memory data

- The [biglm](#) package by Lumley uses incremental computations to offer `lm()` and `glm()` functionality to data sets stored outside of R's main memory.
- The [ff](#) package by Adler et al. offers file-based access to data sets that are too large to be loaded into memory, along with a number of higher-level functions.
- The [bigmemory](#) package by Kane and Emerson permits storing large objects such as matrices in memory (as well as via files) and uses external pointer objects to refer to them. This permits transparent access from R without bumping against R's internal memory limits. Several R processes on the same computer can also share big memory objects.

- A large number of database packages, and database-alike packages (such as [sqldf](#) by Grothendieck and [data.table](#) by Dowle) are also of potential interest but not reviewed here.
- The [MonetDB.R](#) package allows R to access the MonetDB column-oriented, open source database system as a backend.
- The [LaF](#) package provides methods for fast access to large ASCII files in csv or fixed-width format.
- The [bigstatsr](#) package also operates on file-backed large matrices via memory-mapped access, and offers several matrix operations, PCA, sparse methods and more..
- The [disk.frame](#) package leverages several other packages to provide efficient access and manipulation operations for data sets that are larger than RAM.
- The [arrow](#) package offers the portable Apache Arrow in-memory format as well as readers for different file formats which can include support for out-of-memory processing and streaming.

Easier interfaces for Compiled code

- The [inline](#) package by Sklyar et al eases adding code in C, C++ or Fortran to R. It takes care of the compilation, linking and loading of embedded code segments that are stored as R strings.
- The [Rcpp](#) package by Eddelbuettel and Francois offers a number of C++ classes that makes transferring R objects to C++ functions (and back) easier, and the [RInside](#) package by the same authors allows easy embedding of R itself into C++ applications for faster and more direct data transfer.
- The [RcppParallel](#) package by Allaire et al. bundles the [Intel Threading Building Blocks](#) and [TinyThread](#) libraries. Together with [Rcpp](#), RcppParallel makes it easy to write safe, performant, concurrently-executing C++ code, and use that code within R and R packages.
- The [Java](#) package by Urbanek provides a low-level interface to Java similar to the `.Call()` interface for C and C++.
- The [reticulate](#) package by Allaire provides interface to Python modules, classes, and functions. It allows R users to access many high-performance Python packages such as [tensorflow](#) and [tfestimators](#) within R.

Profiling tools

Packages [profvis](#), [proffer](#), [profmem](#), [GUIProfiler](#), [proftools](#), and [aprof](#) summarize and visualize output from the Rprof interface for profiling. The [profile](#) package reads and writes profiling data and converts among file formats such as [pprof](#) by Google and Rprof. The [xrprof](#) command-line tool implements profile sampling for a given R process on Linux or Windows, and it can profile R code alongside compiled code.

CRAN packages

Core: [Rmpi](#), [snow](#).

Regular: [aprof](#), [arrow](#), [batch](#), [BatchExperiments](#), [BatchJobs](#), [batchtools](#), [bcp](#), [BDgraph](#), [biglm](#), [bigmemory](#), [bigstatsr](#), [bnlearn](#), [caret](#), [clustermq](#), [condor](#), [data.table](#), [dclone](#), [disk.frame](#), [doFuture](#), [doMC](#), [doMPI](#), [doRNG](#), [doSNOW](#), [dqrng](#), [drake](#), [ff](#), [flexiblas](#), [flowr](#), [foreach](#), [furr](#), [future](#), [future.apply](#), [future.batchtools](#), [future.callr](#), [gcbd](#), [GPUMatrix](#), [GUIProfiler](#), [h2o](#), [HistogramTools](#), [inline](#), [keras](#), [LaF](#), [latentnet](#), [Matching](#), [mirai](#), [MonetDB.R](#), [mvnfast](#), [nanonext](#), [nlmixr2](#), [OpenCL](#), [orloca](#), [parallelly](#), [parSim](#), [pbapply](#), [pbdMPI](#), [pepper](#), [pls](#), [proffer](#), [profile](#), [profmem](#), [proftools](#), [profvis](#), [pvclust](#), [qsub](#), [randomForestSRC](#), [Rborist](#), [Rcpp](#), [RcppParallel](#), [reticulate](#), [rgenoud](#), [RhpcBLASctl](#), [RInside](#), [rJava](#), [rlecuyer](#), [RProtoBuf](#), [rslurm](#), [rstream](#), [rxode2](#), [rxode2random](#), [Sim.DiffProc](#), [sitmo](#), [snowfall](#), [snowFT](#), [sqldf](#), [ssgraph](#), [targets](#), [tensorflow](#), [tfestimators](#), [tm](#), [varSelRF](#), [xgboost](#).

Related links

- [HPC computing notes by Luke Tierney for HPC class at University of Iowa](#)
- [Mailing List: R Special Interest Group High Performance Computing](#)
- [Schmidberger, Morgan, Eddelbuettel, Yu, Tierney and Mansmann \(2009\) paper on ‘State of the Art in Parallel Computing with R’](#)
- [Luke Tierney’s code directory for pnmath and pnmath0](#)
- [Slurm open-source workload manager](#)
- [Condor project at University of Wisconsin-Madison](#)
- [Parallel Computing in R with sfCluster/snowfall](#)
- [Wikipedia: Message Passing Interface \(MPI\)](#)
- [Wikipedia: Parallel Virtual Machine \(PVM\)](#)
- [Slides from Introduction to High-Performance Computing with R tutorial held in Nov 2009 at the Institute for](#)

[Statistical Mathematics, Tokyo, Japan](#)

- [rgpu project at nbic.nl](#)
- [Magma: Matrix Algebra on GPU and Multicore architectures](#)
- [Parallel R: Data Analysis in the Distributed World](#)
- [High Performance Statistical Computing for Data Intensive Research](#)
- [Rth: Parallel R through Thrust](#)
- [Programming with Big Data in R](#)

Other resources

- Bioconductor Package: [affyPara](#)
- Bioconductor Package: [maanova](#)
- Bioconductor Package: [multtest](#)
- Bioconductor Package: [puma](#)
- R-Forge Project: [biocep-distrib](#)
- GitHub Project: [RHIPE](#)
- Google Code Project: [bugsparell](#)
- Google Code Project: [romp](#)